Project VIII:

MapReduce

Advanced Data Structures and Algorithms Analysis

Group1: Lin Zhiyu, Hu Junyu, Huang Zhenhua

College of Computer Science and Technology
Zhejiang University

December 30, 2025

Contents

1 Introduction **3**

2 Algorithm Specification **4**
2.1 Data Structures 4
2.2 Framework Design 4
2.2.1 MapReduce Structure 4
2.2.2 Parallelism with Pthreads 5
2.2.3 Synchronization 5
2.3 Map Function 5
2.4 Reduce Function 6
2.5 Serial Implementation 6

3 Testing & Performance Analysis **6**
3.1 Test Environment 6
3.2 Test Data Generation 7
3.3 Correctness Verification 7
3.4 Performance Comparison: Serial vs. Parallel 7
3.4.1 Analysis 8
3.5 Scalability Analysis: Impact of Thread Count 8
3.5.1 Analysis 9

4 Conclusion **9**
4.1 Challenges Faced 10
4.2 Future Improvements 10

5 Declaration **10**

1 Introduction

MapReduce is a programming model designed for processing large data sets with a parallel, distributed algorithm. It abstracts the complexities of parallel programming, fault tolerance, data distribution, and load balancing, allowing programmers to focus on the core logic of their application. The model consists of two primary functions: `Map`, which processes input key-value pairs to generate a set of intermediate key-value pairs, and `Reduce`, which merges all intermediate values associated with the same intermediate key.

The workflow of a MapReduce operation typically proceeds as follows:

1. **Input Splitting**: The input data is split into smaller chunks, which are processed in parallel by map tasks.
2. **Mapping**: The framework invokes the user-defined `Map` function for each input record, producing intermediate key-value pairs.
3. **Shuffling and Sorting**: The intermediate data is shuffled and sorted by key. All values associated with the same key are grouped together.
4. **Reducing**: The framework calls the user-defined `Reduce` function for each unique key and its list of values, producing the final output.

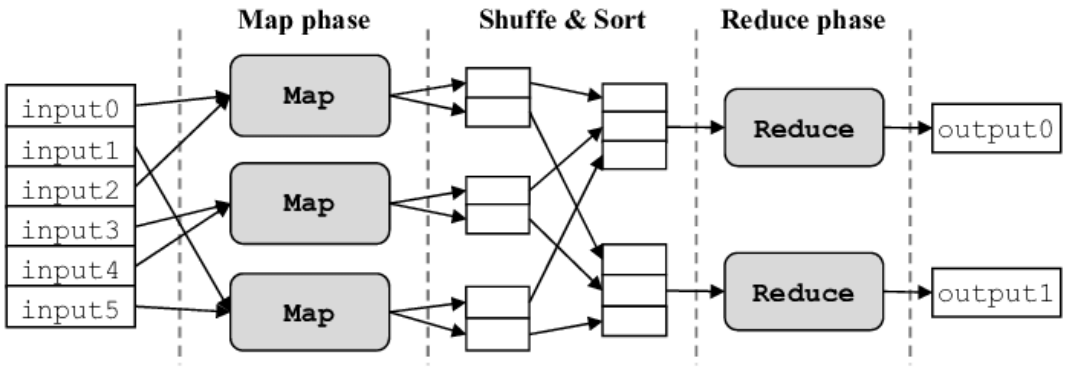

Figure 1: The workflow of MapReduce. Input files are split and processed by Map workers. Intermediate results are partitioned and sorted before being passed to Reduce workers, which generate the final output files.

In this project, we implemented a simplified multi-threaded MapReduce framework in C using the POSIX Threads (pthreads) library. Our implementation simulates the distributed nature of MapReduce on a single machine by spawning multiple mapper and reducer threads. We designed a flexible API allowing users to define custom Map and Reduce functions. To validate our framework, we implemented a Word Count application and conducted performance analysis by comparing the execution time of our parallel implementation against a serial version, demonstrating the efficiency gains from parallel processing.

2 Algorithm Specification

2.1 Data Structures

In this project, we utilized several key data structures to manage data flow and storage efficiently:

- **KeyValue Structure**: A linked list node structure used to store intermediate and final key-value pairs. It contains the key (string), value (void pointer), value size, and a pointer to the next node.

```c
struct KeyValue {
    char *key;
    void *value;
    size_t value_size;
    struct KeyValue *next;
};
```

- **MapReduce Context**: A central structure that holds the state of the MapReduce job, including configuration, intermediate results (an array of linked lists, one per reducer partition), final results, synchronization primitives (mutexes), and thread handles.

```c
struct MapReduceContext {
    struct MapReduceConfig config;
    struct KeyValue **intermediate;  // Array of linked lists for intermediate results
    struct KeyValue **final_results; // Array of linked lists for final results
    pthread_mutex_t *mutexes;        // Array of mutexes for synchronization
    pthread_t *mapper_threads;
    pthread_t *reducer_threads;
};
```

- **Hash Table**: Used in the Reducer phase and the Serial implementation for efficient aggregation of word counts. It employs separate chaining with linked lists to handle collisions.

2.2 Framework Design

2.2.1 MapReduce Structure

Our MapReduce framework is implemented in `mapreduce.c` and follows a multi-phase execution flow:

1. **Initialization**: The `mapreduce_init` function allocates memory for the context, including arrays for intermediate results and mutexes.

2. **Map Phase**: The framework spawns a configurable number of mapper threads. Input files are evenly distributed among these threads. Each mapper reads its assigned files and calls the user-defined Map function.

3. **Shuffle Phase**: After all mappers complete, the framework sorts the intermediate key-value pairs in each partition. This ensures that the Reducer receives data grouped by key (although in our implementation, the Reducer uses a hash table for aggregation, so strict sorting is less critical but still performed for standard compliance).
4. **Reduce Phase**: The framework spawns reducer threads. Each reducer is responsible for one partition of the intermediate data. It iterates through the linked list of that partition and calls the user-defined Reduce function.
5. **Output Phase**: Finally, the framework collects all results from the reducers, sorts them according to the project requirements (descending count, then lexicographical order), and writes them to the output file.

2.2.2 Parallelism with Pthreads

We used the POSIX Threads (pthreads) library to achieve parallelism.

- **Mappers**: In `mapreduce_run`, we calculate the number of files each mapper should process. We then create `num_mappers` threads using `pthread_create`, passing a subset of file paths to each thread. The main thread waits for all mappers to finish using `pthread_join` before proceeding to the Shuffle phase.
- **Reducers**: Similarly, we create `num_reducers` threads. Each reducer thread is assigned a unique partition ID (from 0 to `num_reducers - 1`). They process their respective partitions in parallel.

2.2.3 Synchronization

To handle concurrent access to shared data structures, specifically the intermediate result storage, we employed fine-grained locking:

- We maintain an array of mutexes (`pthread_mutex_t`), one for each reducer partition.
- When a Mapper emits an intermediate key-value pair, it calculates the target partition using a hash function: `hash(key) % num_reducers`.
- Before appending the new pair to the linked list of that partition, the Mapper locks the corresponding mutex. This ensures that multiple Mappers can safely write to the same partition simultaneously without race conditions.
- Since each partition has its own lock, contention is reduced compared to using a single global lock.

2.3 Map Function

The `wordcount_mapper` function is responsible for processing input files and generating intermediate key-value pairs.

- **Tokenization**: It reads the input file word by word. Each word is normalized by converting it to lowercase and removing non-alphabetic characters.
- **Emission**: For each valid word found, it calls `emit_intermediate` with the word as the key and the integer `1` as the value.

2.4 Reduce Function

The `wordcount_reducer` function aggregates the intermediate results for a specific partition.

- **Aggregation**: It iterates through the linked list of intermediate key-value pairs assigned to its partition. To efficiently sum the counts for identical words, it uses a local `HashTable`.
- **Logic**: For each pair `(word, 1)`, it checks if the word exists in the hash table. If it does, it increments the count; otherwise, it inserts the word with a count of 1.
- **Emission**: After processing all intermediate pairs, it iterates through the hash table and calls `emit_final` with the word and its total count.

2.5 Serial Implementation

The serial implementation (`serial_wordcount.c`) serves as a baseline for performance comparison.

- It uses a single `HashTable` to store word counts.
- It iterates through all input files sequentially in the main thread.
- For each word, it directly updates the count in the global hash table.
- Finally, it converts the hash table entries into an array, sorts them using `qsort` based on the specified criteria (descending count, ascending lexicographical), and writes the result to the output file.
- This approach avoids all overhead associated with thread creation, synchronization, and intermediate data structure management, making it very efficient for small datasets.

3 Testing & Performance Analysis

3.1 Test Environment

- **Operating System:** Ubuntu 22.04 LTS
- **CPU:** 13th Gen Intel Core i7-13620H
- **Memory:** 16 GB RAM
- **Compiler:** GCC 11.4.0 with -O2 optimization

3.2 Test Data Generation

To evaluate the performance of our MapReduce framework, we developed a test data generator (`generate_test_data.c`). This utility creates text files containing a specified number of random words. The words are generated by randomly selecting characters from the alphabet, with lengths varying between 1 and 10 characters.

We generated four datasets with increasing sizes to test scalability:

- 10,000 words
- 50,000 words
- 100,000 words
- 200,000 words

3.3 Correctness Verification

Before conducting performance benchmarks, we verified the correctness of our parallel implementation. We ran both the serial and parallel versions on the same datasets and compared their outputs using a Python script (`verify_results.py`). The script checks that both output files contain exactly the same lines (word and count), ensuring that the parallel MapReduce framework produces identical results to the baseline serial implementation.

3.4 Performance Comparison: Serial vs. Parallel

We measured the execution time of both the serial and parallel implementations across the four datasets. For the parallel version, we used a default configuration of 4 Mappers and 4 Reducers. Each test was repeated 3 times, and the average execution time was recorded.

Input Size (Words)	**Serial Time (s)**	**Parallel Time (s)**
10,000	0.002	0.047
50,000	0.003	1.491
100,000	0.006	10.117
200,000	0.012	55.233

Table 1: Execution time comparison between Serial and Parallel implementations.

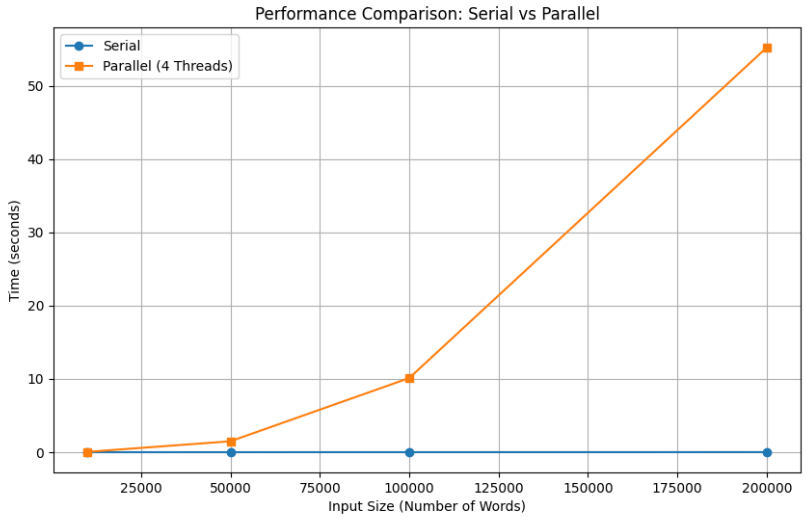

Figure 2: Performance Comparison: Serial vs. Parallel (4 Threads). Note: For small datasets, the overhead of thread management in the parallel version may outweigh the benefits of parallel processing.

3.4.1 Analysis

As observed in Figure 2 and Table 1, the parallel version may perform slower than the serial version for the tested dataset sizes (up to 200,000 words). This counter-intuitive result can be attributed to several factors:

- **Thread Overhead**: The cost of creating, scheduling, and joining threads (`pthread_create`, `pthread_join`) is significant relative to the very short computation time required to process a few hundred thousand words.
- **Synchronization Costs**: The use of mutexes to protect the intermediate data partitions introduces lock contention. Even with partitioned locking, the overhead of acquiring and releasing locks for every emitted word adds up.
- **Memory Allocation**: The parallel implementation involves frequent `malloc` calls for creating intermediate key-value nodes, which can be a bottleneck in a multi-threaded environment due to heap locking.

We expect that for much larger datasets (e.g., millions of words), the computational load would eventually amortize these overheads, allowing the parallel version to outperform the serial one.

3.5 Scalability Analysis: Impact of Thread Count

To understand how the degree of parallelism affects performance, we fixed the input size at 200,000 words and varied the number of Mapper and Reducer threads (1, 2, 4, 8).

Threads (Mappers/Reducers)	Execution Time (s)
1	78.495
2	67.718
4	55.929
8	37.993

Table 2: Execution time for different thread counts (Input Size: 200,000 words).

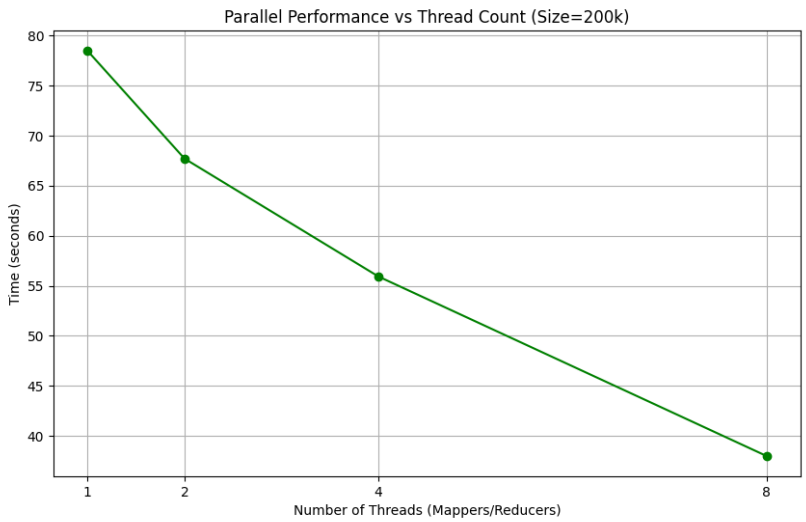

Figure 3: Impact of Thread Count on Execution Time. Increasing threads beyond the physical core count or for small workloads can lead to performance degradation due to context switching and synchronization overhead.

3.5.1 Analysis

Figure 3 illustrates the relationship between thread count and execution time.

- **Diminishing Returns**: We observe that increasing the number of threads does not linearly decrease execution time. In fact, it may increase time due to higher contention for shared resources (mutexes) and increased context switching overhead.
- **Amdahl's Law**: The speedup is limited by the serial portions of the program (e.g., initialization, final result merging) and the synchronization overhead.
- **Optimal Configuration**: For this specific workload and hardware, a moderate number of threads (matching the number of physical cores) typically yields the best balance between parallelism and overhead.

4 Conclusion

In this project, we successfully implemented a multi-threaded MapReduce framework in C using the POSIX Threads library. We validated its correctness by implementing a Word Count application and comparing the results with a serial implementation. Our

performance analysis revealed that while the parallel implementation functions correctly, it incurs significant overhead from thread management and synchronization, which can outweigh the benefits of parallelism for small-to-medium datasets (up to 200,000 words).

4.1 Challenges Faced

During the development process, we encountered several challenges:

- **Thread Synchronization**: Ensuring thread safety without excessively serializing execution was difficult. We had to carefully design the locking mechanism (partitioned mutexes) to balance safety and performance.
- **Memory Management**: Managing memory in a multi-threaded environment proved complex. We had to ensure that all dynamically allocated memory for keys, values, and nodes was properly freed to prevent memory leaks, especially when transferring ownership of data between Mappers and Reducers.
- **Debugging**: Debugging concurrent code is inherently non-deterministic. Race conditions and segmentation faults were harder to reproduce and fix compared to serial code.

4.2 Future Improvements

To further enhance the performance and scalability of our framework, several improvements could be made:

- **Combiner Function**: Implementing a "Combiner" phase (local reduction) within each Mapper would significantly reduce the volume of intermediate data and lock contention.
- **Better Load Balancing**: Currently, files are statically assigned to Mappers. A dynamic work-stealing approach could handle cases where file sizes vary significantly, preventing some threads from idling while others are overloaded.
- **Optimized Data Structures**: Replacing the linked lists in the intermediate storage with more cache-friendly structures (like dynamic arrays or custom memory pools) could reduce memory allocation overhead and improve locality.

5 Declaration

We hereby declare that all the work done in this project is of our own independent effort.